

Smart Document Q&A – Project Summary

I built a Retrieval-Augmented Generation (RAG) application that allows users to upload a PDF document and ask natural language questions about its contents. The system retrieves the most relevant sections of the document and uses a local Large Language Model (Llama 3.2 via Ollama) to generate context-aware answers.

Technologies Used

- Python
- FastAPI
- Streamlit
- Sentence Transformers
- FAISS
- Ollama (Llama 3.2)

Architecture

1. Upload PDF through Streamlit.
2. FastAPI reads and extracts the document text.
3. The document is split into chunks.
4. Sentence Transformers generate embeddings for each chunk.
5. Embeddings are indexed using FAISS.
6. When a user asks a question, the query is embedded and searched against the FAISS index.
7. The most relevant chunks are passed to the LLM to generate the final answer.
8. The answer is displayed in the Streamlit chat interface.

Trade-offs

- I chose FAISS for efficient similarity search because it significantly improves retrieval speed compared to manually comparing embeddings.
- I used Ollama with a locally running Llama 3.2 model to avoid dependency on paid cloud APIs and keep the application fully local.
- The current implementation processes one document at a time, which keeps the architecture simple but limits multi-document querying.

What I Would Do Differently

With more time, I would:

- Support multiple uploaded documents simultaneously.
- Maintain conversational memory so follow-up questions understand previous context.
- Store FAISS indexes persistently instead of rebuilding them after each upload.
- Highlight the exact paragraph used to generate the answer.